

CONCEPTION ET DEPLOIEMENT D'UNE ARCHITECTURE MICROSERVICES AVEC DOCKER ET KUBERNETES

Projet Programmation Déployée

Marine GIMENEZ

Hanna SYSOIEVA

MLSD 1 FA

2025/2026

Table des matières

Introduction	2
Description d'application	2
Déploiement	3
1. Création et conteneurisation du premier microservice	3
2. Déploiement du premier microservice sur Kubernetes	3
3. Création et déploiement du deuxième microservice	4
4. Mise en place d'un Ingress	5
5. Intégration d'une base de données MySQL	7
6. Sécurisation du cluster avec RBAC	8
Liens & Google labs.....	9

Introduction

Dans le cadre de ce projet de programmation distribuée, on a développé une application basée sur une architecture microservices. L'objectif est de mettre en œuvre différentes technologies modernes telles que Docker et Kubernetes, en respectant une approche progressive, c'est-à-dire partir d'un service simple, puis évoluer vers plusieurs services interconnectés.

On a choisi de développer deux microservices distincts, chacun encapsulé dans un conteneur Docker, puis déployés sur un cluster Kubernetes via Minikube.

Vous trouver les démarches suivis dans ce rapport.

Description d'application

L'idée derrière notre application est de permettre la création de post-cards personnalisées illustrées par un cœur, que l'utilisateur peut envoyer à une personne afin de transmettre un message de manière simple et agréable.

Lorsque l'utilisateur ouvre l'application, il accède à une page d'accueil sur laquelle il peut saisir le prénom du destinataire, entrer un message personnalisé et choisir la couleur du cœur. Une fois ces informations renseignées, il lui suffit de cliquer sur le bouton « Generate » pour générer automatiquement la carte. L'utilisateur est alors redirigé vers une nouvelle page affichant le visuel final de la post-card avec le cœur et le message.

Par exemple, on peut générer une carte destinée à Marine avec le message « Thanks a lot », qui sera affichée avec le design choisi.

Déploiement

1. Création et conteneurisation du premier microservice

On a commencé par développer un premier microservice nommé heart.

On a créé une image Docker à partir de notre application avec la commande suivante :

```
docker build -t heart:v1 .
```

Ensuite, on a lancé un conteneur basé sur cette image :

```
docker run -d -p 4444:5000 heart:v1
```

Cette commande permet d'exécuter le service en local, avec un mapping du port 5000 du conteneur vers le port 4444 de la machine hôte.

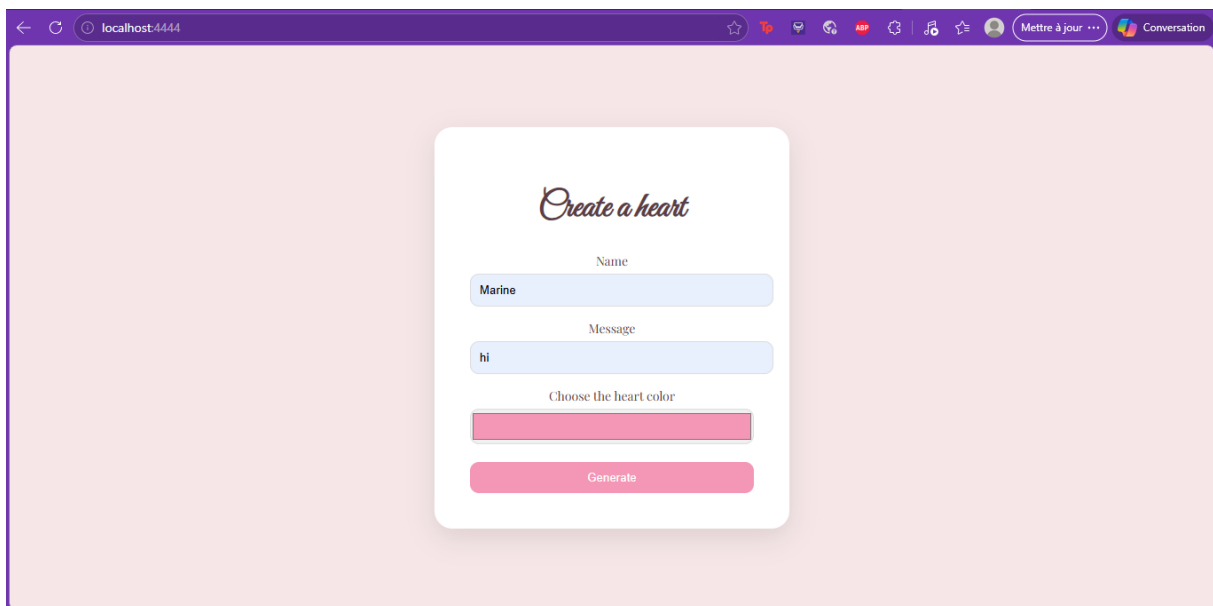


Figure 1. Interface de l'application du microservice heart en local

Afin de rendre l'image accessible pour Kubernetes, on l'a publiée sur Docker Hub :

```
docker tag heart:v1 gimenezm/heart:v1
```

```
docker login
```

```
docker push gimenezm/heart:v1
```

On a donc renommé l'image puis on l'a poussée vers notre repository Docker Hub.

2. Déploiement du premier microservice sur Kubernetes

On a utilisé Minikube pour créer un cluster Kubernetes local :

```
minikube start
```

```
kubectl get nodes
```

Cette commande permet de vérifier que le cluster est bien démarré et que les nœuds sont disponibles.

```
PS C:\Users\dolph\Documents\Master\M1\Programmation distribuée\projet\heart> kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	13d	v1.35.1

Figure 2. Résultat de la commande `kubectl get nodes`

On a ensuite déployé notre microservice sur Kubernetes :

```
kubectl apply -f heart-deployment.yml
kubectl apply -f heart-service-clusterip.yml
```

Le fichier deployment permet de définir le nombre de réplicas et le conteneur utilisé et le fichier service (ClusterIP) permet d'exposer le microservice à l'intérieur du cluster.

3. Création et déploiement du deuxième microservice

On a répété le même processus pour un deuxième microservice nommé titre.

```
docker build -t titre:v1 .
docker run -d -p 4445:5000 titre:v1
```

Ce service est accessible localement sur un port différent afin d'éviter les conflits.

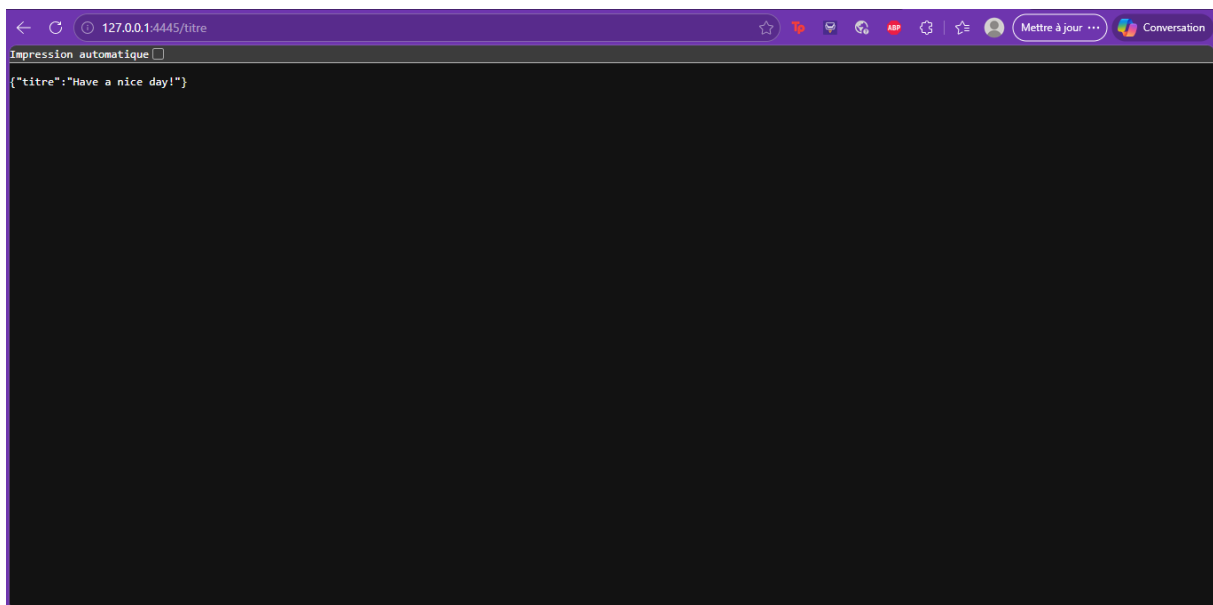


Figure 3. Interface de l'application du microservice titre en local

Ensuite on l'a publié sur Docker Hub et l'a déployé sur Kubernetes.

```
docker tag titre:v1 gimenezm/titre:v1
docker login
docker push gimenezm/titre:v1

minikube start
kubectl get nodes
```

```
PS C:\Users\dolph\Documents\Master\M1\Programmation distribuée\projet\heart> kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready    control-plane  13d   v1.35.1
```

Figure 4. Vérification des nœuds Kubernetes

Puis :

```
kubectl apply -f titre-deployment.yml
kubectl apply -f titre-service-clusterip.yml
```

On a donc déployé le deuxième microservice et créé un service interne pour permettre sa communication dans le cluster.

4. Mise en place d'un Ingress

Afin de permettre l'accès aux microservices depuis l'extérieur, on a configuré un Ingress.

```
minikube addons enable ingress
kubectl get pods -n ingress-nginx
```

On a activé le contrôleur Ingress basé sur NGINX et vérifié que les pods associés sont en cours d'exécution.

```
kubectl apply -f ingress.yml
kubectl get ingress
```

Le fichier ingress.yml permet de définir des règles de routage vers les différents services.

Sur Windows, on a modifié le fichier hosts :

c:\windows\system32\drivers\etc\hosts

Et on a ajouté la ligne suivante :

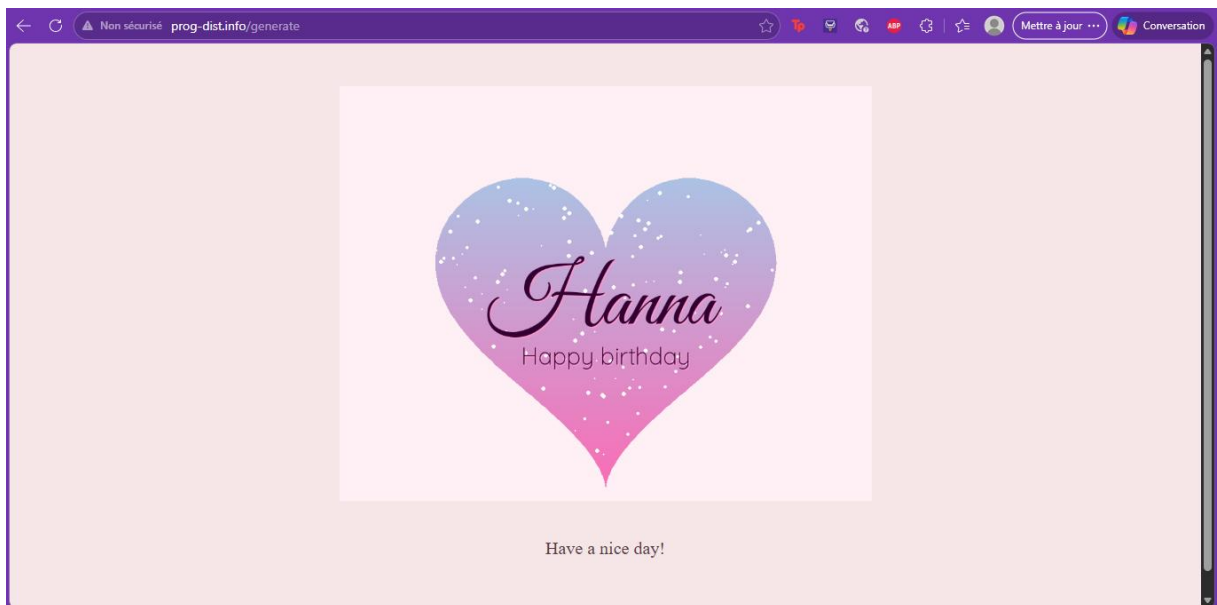
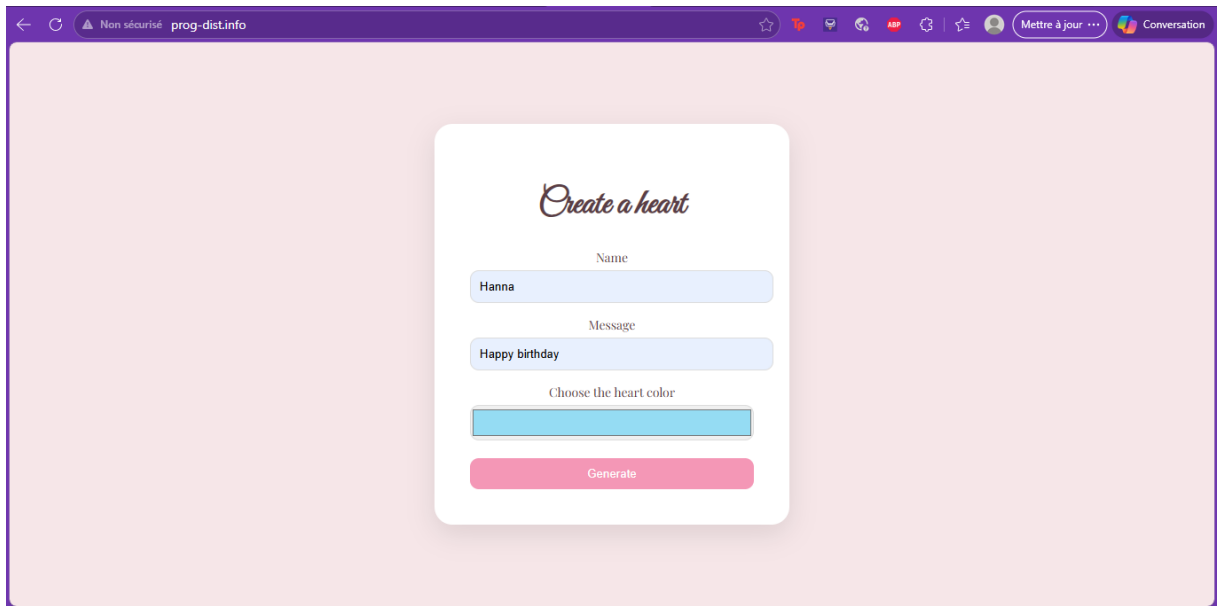
127.0.0.1 prog-dist.info

On a ensuite activé les services nécessaires :

```
minikube addons enable ingress-dns
minikube tunnel
```

Puis on a accédé à l'application via :

http://prog-dist.info



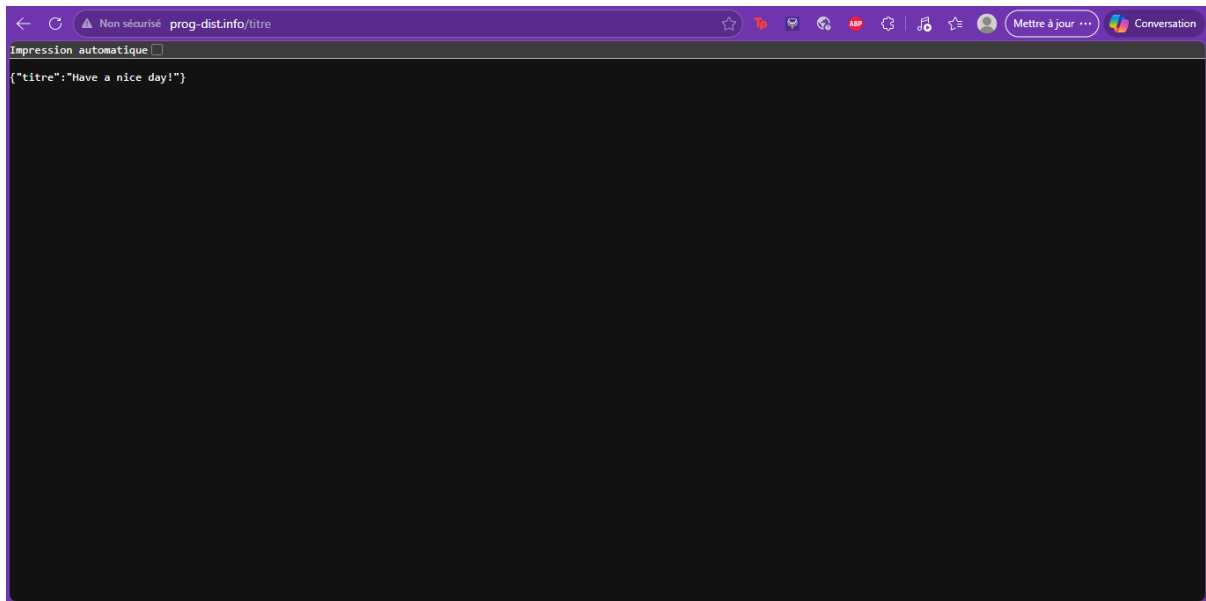


Figure 5. Accès aux différentes pages de l'application via Ingress

5. Intégration d'une base de données MySQL

Afin d'ajouter de la persistance à notre application, on a intégré une base de données MySQL dans notre architecture Kubernetes.

L'objectif était que, lorsqu'un utilisateur clique sur le bouton "generate" dans la première page de l'application, les données saisies (prénom et message) soient enregistrées dans la base de données.

On a commencé par adapter les fichiers de configuration YAML fournis pour MySQL, notamment en modifiant le mot de passe par défaut.

On a ensuite déployé les différents composants nécessaires :

```
kubectl apply -f mysql-secret.yaml
kubectl apply -f mysql-storage.yaml
kubectl apply -f mysql-configmap.yaml
kubectl apply -f mysql-deployment.yaml
kubectl apply -f mysql-serviceNodePort.yaml
```

On a mis en place une chaîne complète de stockage :

MySQL → PersistentVolumeClaim → PersistentVolume → Disk

Cela permet de garantir que les données sont conservées même si le pod est recréé.

On a vérifié le bon déploiement avec les commandes suivantes :

```
kubectl get secrets
kubectl get PersistentVolumes
kubectl get PersistentVolumeClaims
kubectl get deployments
```



```
kubectl get services
```

```
kubectl get pods
```

Pour vérifier le bon fonctionnement de la base, on s'est connecté directement au pod MySQL :

```
kubectl exec --stdin --tty mysql-8f5d78b77-5rjrz -- mysql -ptest1234
```

On a ensuite exécuté une requête SQL afin de vérifier que les données générées depuis l'application étaient bien enregistrées.

```
mysql> select * from db_exemple.MESSAGE;
+-----+-----+-----+
| id | name  | message      |
+-----+-----+-----+
| 1  | Marine | Be Happy !   |
| 2  | Marine | Thanks a lot |
| 3  | Hanna  | Happy birthday |
| 4  | Hanna  | Happy birthday |
| 5  | Hanna  | Happy birthday |
+-----+-----+-----+
```

Figure 6. Résultat de la requête SQL montrant les données sauvegardées

On a également utilisé la commande suivante :

```
minikube mount ./mnt/data
```

Cela permet de monter un répertoire local dans le cluster Kubernetes afin de faciliter l'accès aux fichiers et au stockage.

6. Sécurisation du cluster avec RBAC

Afin de renforcer la sécurité de notre cluster Kubernetes, on a mis en place un système de contrôle d'accès basé sur les rôles.

On a défini les ressources suivantes :

```
kubectl apply -f serviceaccount.yaml
```

```
kubectl apply -f role.yaml
```

```
kubectl apply -f rolebinding.yaml
```

On a vérifié leur création avec :

```
kubectl get serviceaccount
```

```
kubectl get role
```

```
kubectl get rolebinding
```

On a ensuite testé les droits associés au service account avec les commandes suivantes :

```
kubectl auth can-i get pods --as=system:serviceaccount:default:heart-user
```

```
kubectl auth can-i list services --as=system:serviceaccount:default:heart-user
```

```
kubectl auth can-i delete pods --as=system:serviceaccount:default:heart-
user
kubectl auth can-i create deployments --
as=system:serviceaccount:default:heart-user
```

Ces tests permettent de vérifier précisément quelles actions sont autorisées ou non.

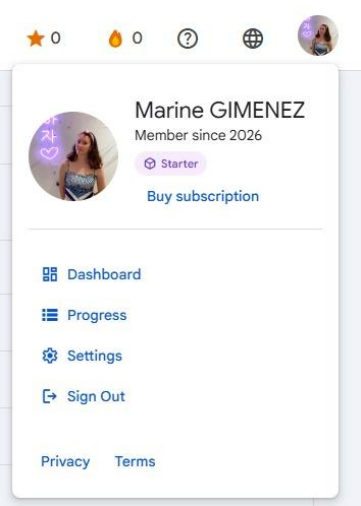
Liens & Google labs

Vous pouvez trouver le projet complet via ce lien github :

[lilriona/projet-prog-dist](https://github.com/lilriona/projet-prog-dist)

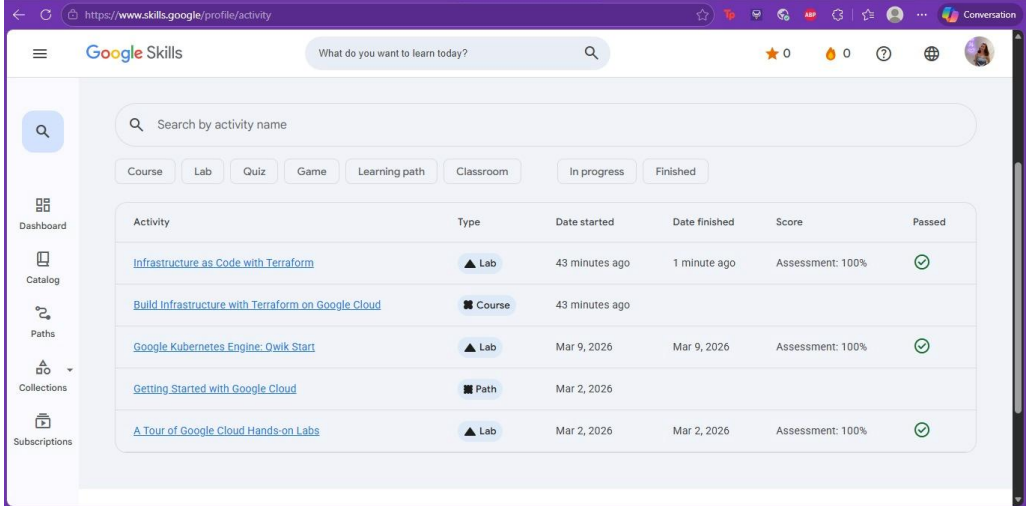
Ci-joint également les captures d'écran des Google labs :

Marine GIMENEZ :



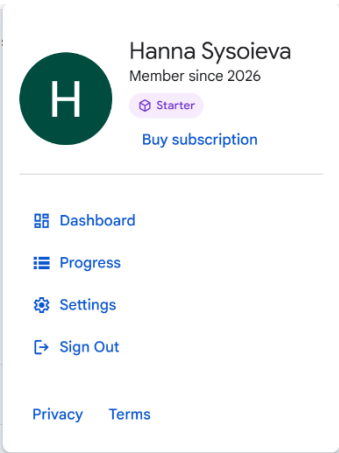
Marine GIMENEZ
Member since 2026
Starter
Buy subscription

- Dashboard
- Progress
- Settings
- Sign Out
- Privacy Terms



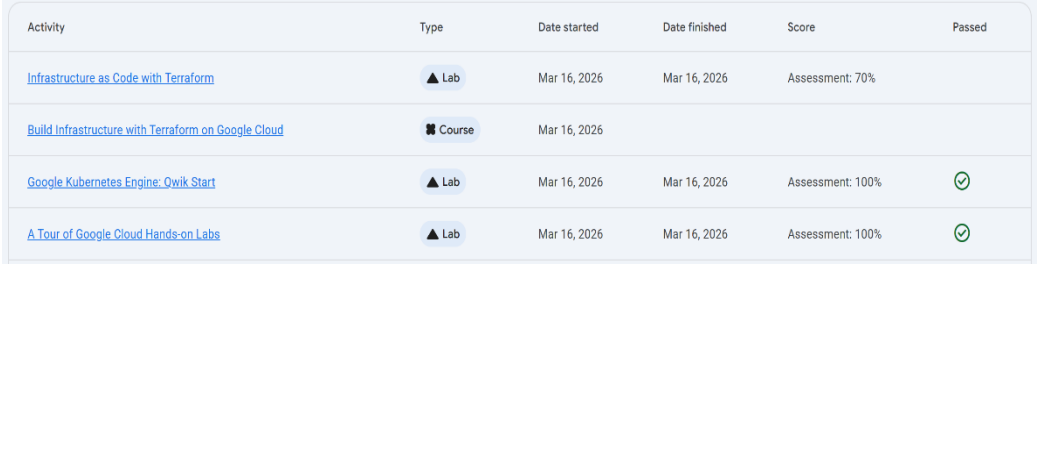
Activity	Type	Date started	Date finished	Score	Passed
Infrastructure as Code with Terraform	Lab	43 minutes ago	1 minute ago	Assessment: 100%	✓
Build Infrastructure with Terraform on Google Cloud	Course	43 minutes ago			
Google Kubernetes Engine: Qwik Start	Lab	Mar 9, 2026	Mar 9, 2026	Assessment: 100%	✓
Getting Started with Google Cloud	Path	Mar 2, 2026			
A Tour of Google Cloud Hands-on Labs	Lab	Mar 2, 2026	Mar 2, 2026	Assessment: 100%	✓

Hanna SYSOIEVA :



Hanna Sysoieva
Member since 2026
Starter
Buy subscription

- Dashboard
- Progress
- Settings
- Sign Out
- Privacy Terms



Activity	Type	Date started	Date finished	Score	Passed
Infrastructure as Code with Terraform	Lab	Mar 16, 2026	Mar 16, 2026	Assessment: 70%	
Build Infrastructure with Terraform on Google Cloud	Course	Mar 16, 2026			
Google Kubernetes Engine: Qwik Start	Lab	Mar 16, 2026	Mar 16, 2026	Assessment: 100%	✓
A Tour of Google Cloud Hands-on Labs	Lab	Mar 16, 2026	Mar 16, 2026	Assessment: 100%	✓